

ECE 5755 Lab 0: Sorting and Top-Down Analysis

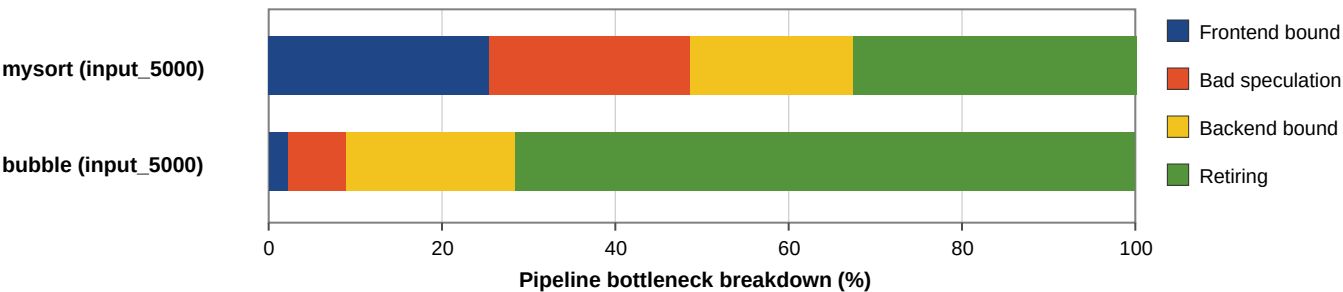
5,000-element input on Intel Xeon Silver 4514Y (Sapphire Rapids)

Implementation.

I implemented an in-place, recursive top-down mergesort in `mysort.c`. Each recursive call divides the array, sorts both halves, and merges them using temporary left and right arrays; the `<=` comparison preserves stability. The wrapper returns the same sorted input pointer. This has $O(n \log n)$ time complexity and $O(n)$ temporary merge storage. The output passed the supplied sortedness check. The merge-sort structure was adapted from GeeksforGeeks, "Merge Sort," <https://www.geeksforgeeks.org/dsa/merge-sort/> (accessed September 8, 2026).

Top-down profiling.

I ran `pmu-tools/toplev.py` with `-l1 -v --no-desc --force-cpu spr` on the ECE 5755 compute node. The horizontal bar chart below compares the Level-1 pipeline bottleneck breakdown (%) for both implementations.



Implementation	Frontend	Bad spec.	Backend	Retiring
Bubble sort	2.3%	6.7%	19.5%	71.5%
Mergesort	25.4%	23.3%	18.7%	32.7%

Observations.

Bubble sort was dominated by retiring (71.5%), indicating that most measured slots completed useful instructions. Mergesort shifted toward frontend bound (25.4%) and bad speculation (23.3%), with only 32.7% retiring. Its backend-bound percentage was similar to bubble sort (18.7% versus 19.5%). The different profile is consistent with mergesort recursion, merge control flow, and temporary-array accesses, although the percentages describe pipeline behavior rather than algorithmic complexity alone. Both runs reported 100% counter multiplexing coverage.

Issues.

Profiling from the login node was blocked by `perf_event_paranoid=4`; running inside an `srun` allocation on the instructional compute node resolved access. `toplev` also warned about the NMI watchdog and could not open an uncore file, but it still produced the core Level-1 results. The final packaged source leaves the verification call commented out as required for profiling.